Práctica 4: Configuración de Google Stitch MCP en VS Code

Oct 15, 2026 · Britney Mishel Félix López

Datos generales

Campo	Dato
Alumna	Britney Mishel Félix López
Carrera	Ingeniería en Entornos Virtuales y Negocios Digitales
Universidad	Universidad Tecnológica Metropolitana
Cuatrimestre y grupo	7.º cuatrimestre, grupo 7-E
Materia y docente	(completar)
Práctica	4. Configuración de Google Stitch MCP en VS Code

Introducción y requisitos

El objetivo de la práctica es conectar Google Stitch con VS Code mediante MCP, para que el agente de IA lea los diseños de Stitch y los convierta en vistas Blade dentro de un proyecto Laravel. En mi caso las vistas son las tres que pidió el docente: **registro de usuario**, **listar usuarios** y **editar perfil**, aplicadas al proyecto `saludo-app` de la Práctica 3.

Requisito del manual	Cómo lo cumplí
VS Code actualizado con soporte MCP	VS Code 1.139
Cuenta de GitHub con acceso a Copilot	GitHub Copilot Chat en modo Agent
Cuenta de Google con Stitch y un proyecto con pantallas	Proyecto **Panel de Usuarios Saludo-App** con tres pantallas: listado de usuarios, registro de usuarios y editar perfil
Proyecto Laravel abierto en VS Code	`saludo-app` (Laravel 13 con SQLite), abierto con Archivo → Abrir carpeta
Tailwind CSS con Vite (recomendado)	Tailwind CSS 4 + Vite

Como no tenía proyectos en Stitch, primero creé el proyecto en stitch.withgoogle.com y le pedí en su chat las tres pantallas para `saludo-app`, con el sistema de diseño "Warm Architectural Craft" y el color terracota `#94452C` como color principal.

Paso 1: Instalar GitHub Copilot Chat

Actualicé VS Code e instalé la extensión **GitHub Copilot Chat** desde Extensiones (Ctrl+Shift+X). Abrí el chat con Ctrl+Alt+I y verifiqué que existe el modo **Agent**, que es el único que puede usar herramientas MCP.

Paso 2: Generar la API key de Stitch

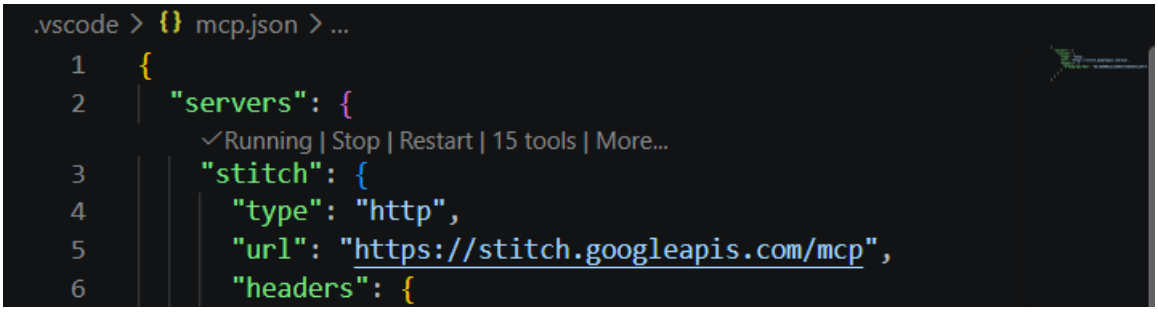
En Stitch entré a mi foto de perfil → **Stitch Settings** → API Keys y usé **Create API Key**. Seguí las buenas prácticas del manual: la clave no queda escrita en ningún archivo del proyecto ni se sube a Git, y al terminar la práctica la elimino y genero una nueva.

Paso 3: Configurar el servidor MCP en VS Code

En la raíz de `saludo-app` creé la carpeta `.vscode` y dentro el archivo `mcp.json`:

```
{
  "inputs": [
    {
      "type": "promptString",
      "id": "stitch-api-key",
      "description": "API key de Google Stitch",
      "password": true
    }
  ],
  "servers": {
    "stitch": {
      "type": "http",
      "url": "https://stitch.googleapis.com/mcp",
      "headers": {
        "X-Goog-Api-Key": "${input:stitch-api-key}"
      }
    }
  }
}
```

Usé la clave `servers` (no `mcpServers`). Además del contenido del manual agregué el bloque `inputs`, porque sin él VS Code marcaba el error *"Variable 'stitch-api-key' must be defined in an 'inputs' section"*. Con el bloque, al dar **Start** VS Code pidió la API key y el servidor pasó a **Running** con **15 herramientas**:



Evidencia del paso 3: el servidor `stitch` en estado Running con 15 herramientas.

Paso 4: Listar proyectos de Stitch

Con el servidor en Running abrí el chat en modo **Agent** y escribí:

> *Usa el servidor MCP de Stitch: lista mis proyectos, luego lista las pantallas del proyecto "Panel de Usuarios Saludo-App" (listado de usuarios, registro de usuarios y editar perfil) y dame un resumen de cada una. No modifiques ningún archivo del proyecto.*

Copilot pidió permiso para ejecutar las herramientas de Stitch, acepté, y encontró el proyecto **Panel de Usuarios Saludo-App**, que es con el que trabajé.

Paso 5: Listar las vistas de un proyecto

En la misma conversación Copilot listó las pantallas del proyecto y las describió:

Pantalla de Stitch	Descripción que dio Copilot
Listado de usuarios	Vista principal para consultar el directorio de usuarios
Registro de usuarios	Formulario de alta con nombre, correo, contraseña y confirmación; incluye indicador de seguridad de contraseña
Editar perfil	Pantalla para actualizar los datos de un perfil existente

Paso 6: Aplicar una vista en el proyecto Laravel

6.1 Preparar Tailwind

`saludo-app` ya tenía Tailwind CSS 4 con Vite. Siguiendo el consejo del manual, primero pasé los colores y tipografías del proyecto de Stitch al `@theme` de `resources/css/app.css`, para que las tres vistas compartan la misma base. El layout `resources/views/layouts/app.blade.php` carga los assets con `@vite(['resources/css/app.css', 'resources/js/app.js'])`.

6.2 Pedir la conversión

El docente dejó elegir el agente (Copilot, Antigravity o Claude). La conversión la hice con Claude, a partir de las pantallas de Stitch y convirtiendo **una pantalla por petición**, como recomienda el manual:

> *Toma la pantalla "Listado de usuarios" de Stitch y conviértela en `resources/views/usuarios/index.blade.php`, que extienda `layouts/app.blade.php` con `@extends` y `@section('content')`. Muestra los usuarios de la tabla `users` con búsqueda y paginación y agrega la ruta GET `/usuarios` con nombre `usuarios.index`.*

> *Toma la pantalla "Registro de usuarios" de Stitch y conviértela en `resources/views/usuarios/registro.blade.php`. El formulario debe usar `@csrf`, `action="{{ route('usuarios.store') }}"`, `method="POST"`, `old()` y `@error`. Agrega las rutas `usuarios.registro` y `usuarios.store`.*
>
> *Toma la pantalla "Editar perfil" de Stitch y conviértela en `resources/views/usuarios/editar.blade.php` para editar nombre, correo y contraseña. Usa `@csrf`, `@method('PUT')`, `old()` y `@error`, y agrega las rutas `usuarios.editar` y `usuarios.update`.*

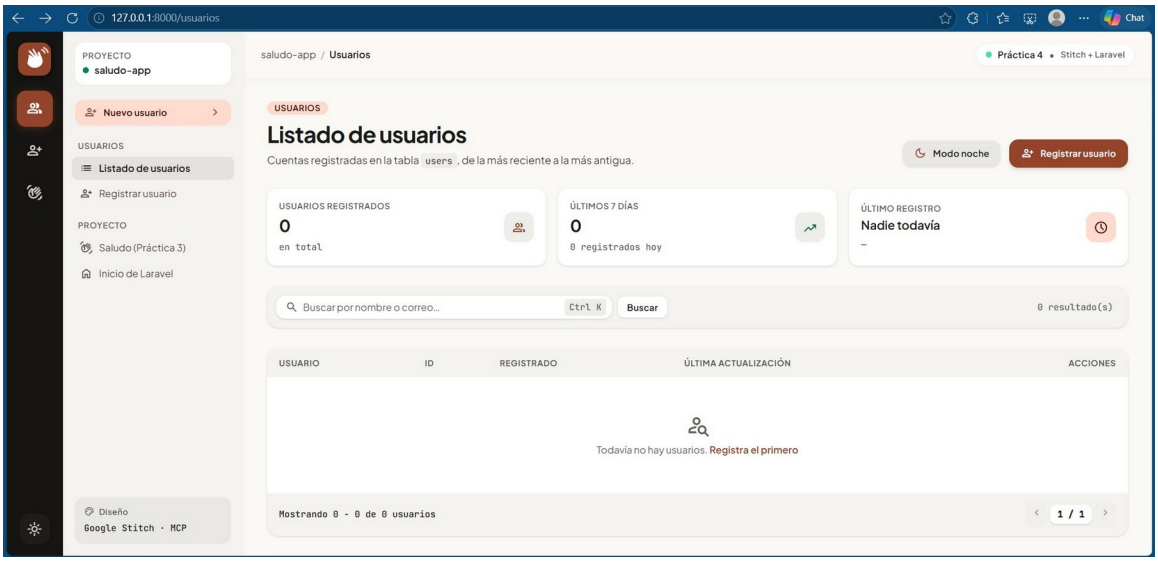
Así se cumplió cada indicación del paso 6.2:

Indicación del manual	Cómo quedó
1. Vista que extienda `layouts/app.blade.php` con `@extends` y `@section('content')`; layout con `<head>`, fuentes y `@vite`	Las tres vistas en `resources/views/usuarios/` extienden el layout
2. Tailwind con las clases del diseño, sin el script del CDN	Tailwind se compila con Vite desde `resources/css/app.css`
3. Botones, inputs y demás en componentes Blade	`resources/views/components/ui/` (button, input, card, badge, avatar…) y `components/layout/`
4. Formulario con `@csrf`, `route()`, `POST`, `old()` y `@error`	`registro.blade.php` y `editar.blade.php`
5. Imágenes con `asset('images/...')`	Logo en `public/images/logo-saludo.svg`; los avatares muestran iniciales
6. Rutas en `routes/web.php`	`usuarios.index`, `usuarios.registro`, `usuarios.store`, `usuarios.editar`, `usuarios.update`

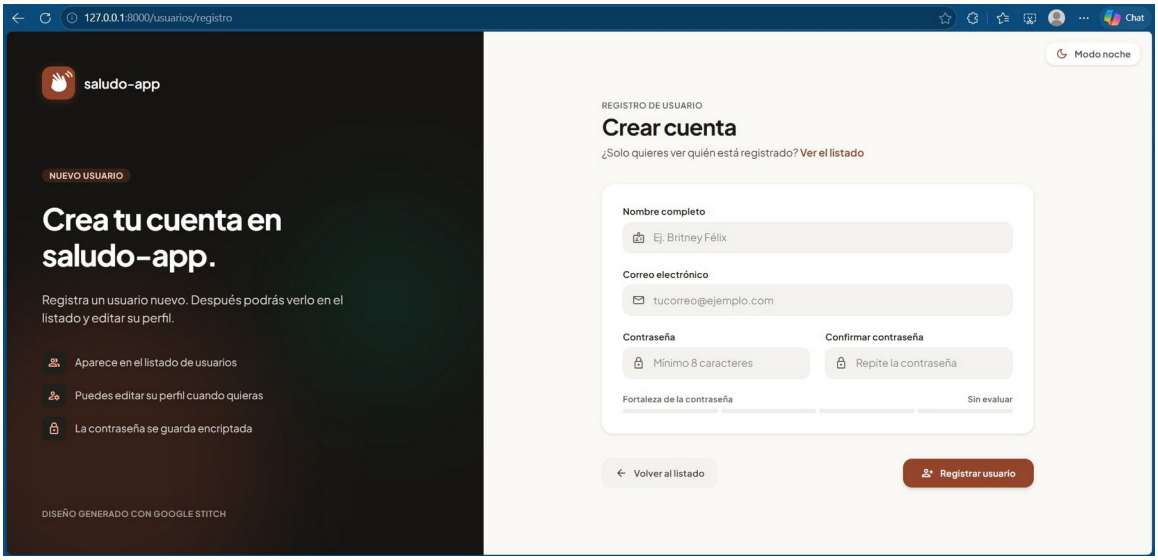
El controlador es `app/Http/Controllers/UsuarioController.php` (index, create, store, edit, update). La contraseña se guarda encriptada por el cast `hashed` del modelo `User`, y los mensajes de validación están en español en `app/Support/Validacion.php`.

6.3 Revisar y probar

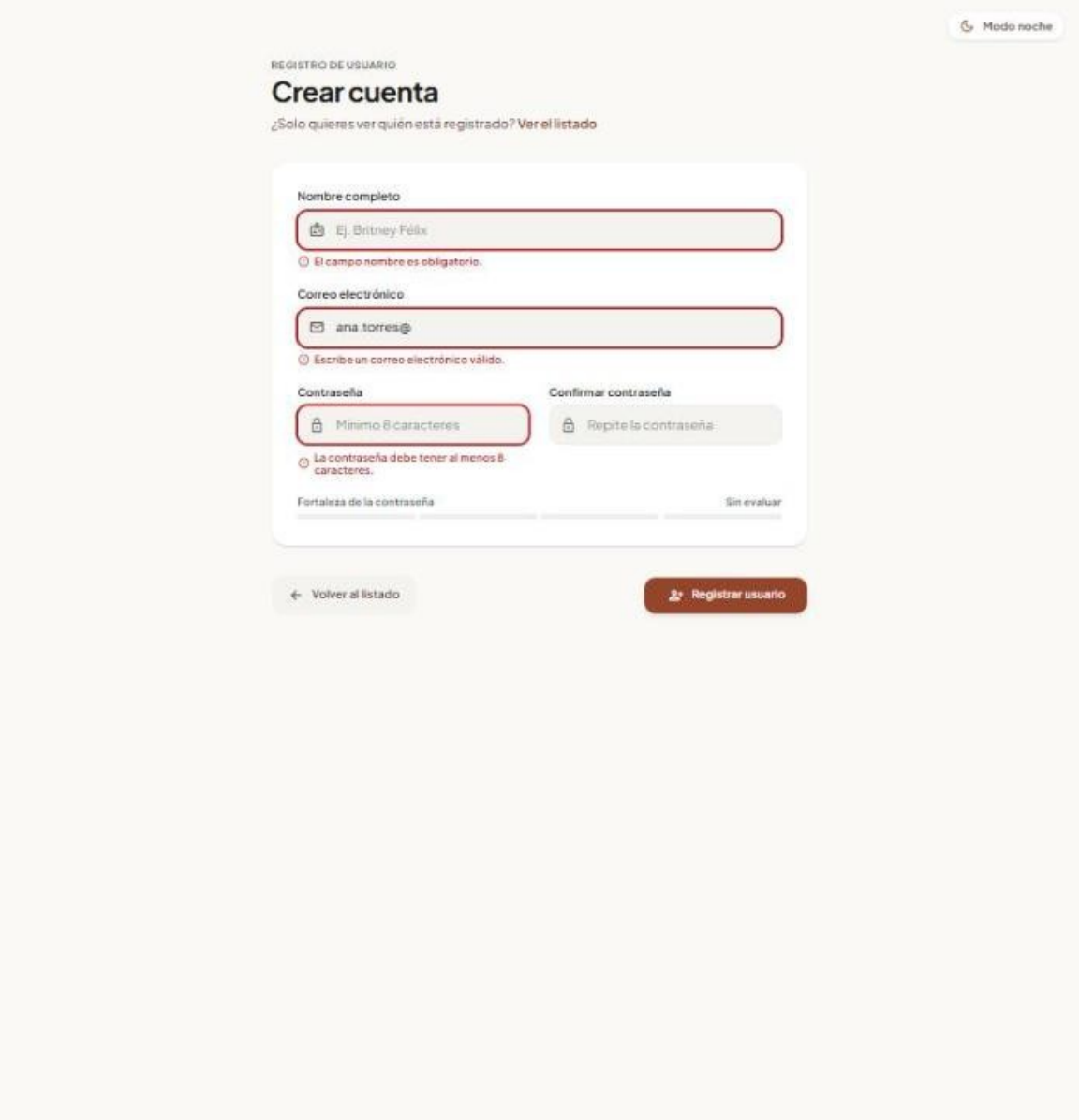
Levanté el servidor con `php artisan serve` y abrí http://127.0.0.1:8000/usuarios. Para no tener que escribir comandos, dejé los estilos compilados en `public/build` y el archivo `iniciar.bat`, que levanta el servidor y abre el navegador con doble clic. Estas capturas son del proyecto corriendo en mi computadora:



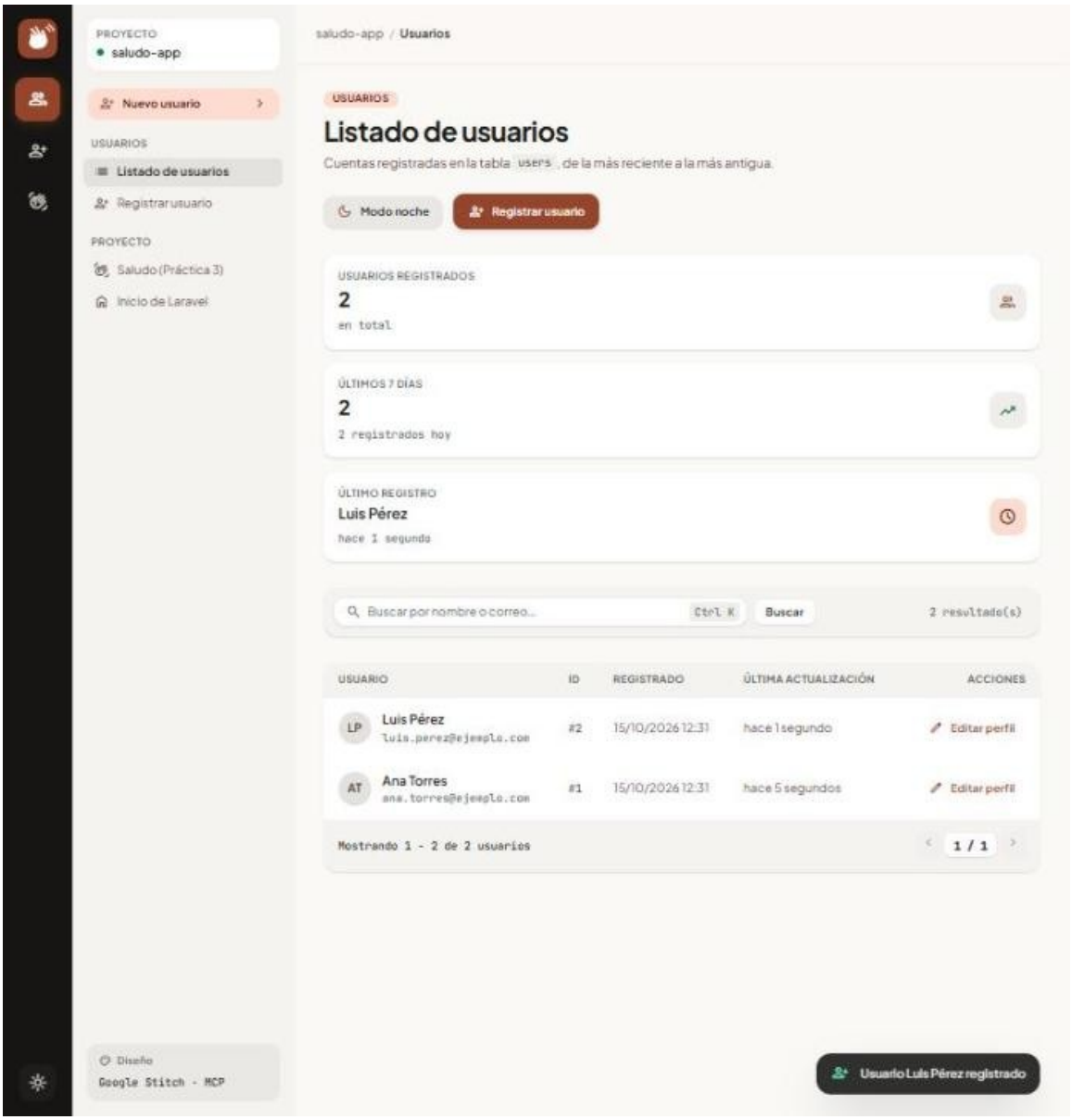
Captura 1. Listado de usuarios antes de registrar a alguien.



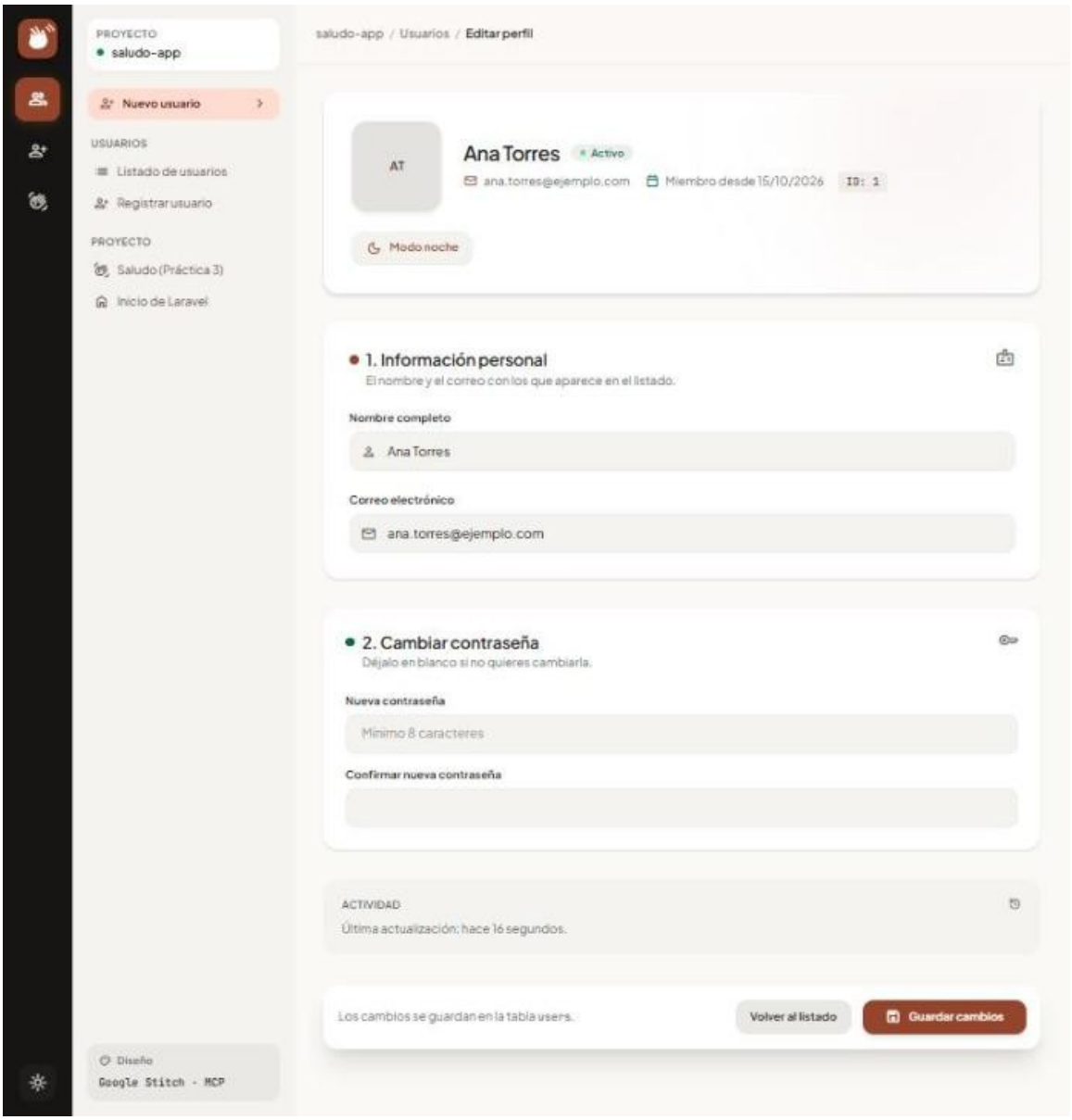
Captura 2. Registro de usuario.



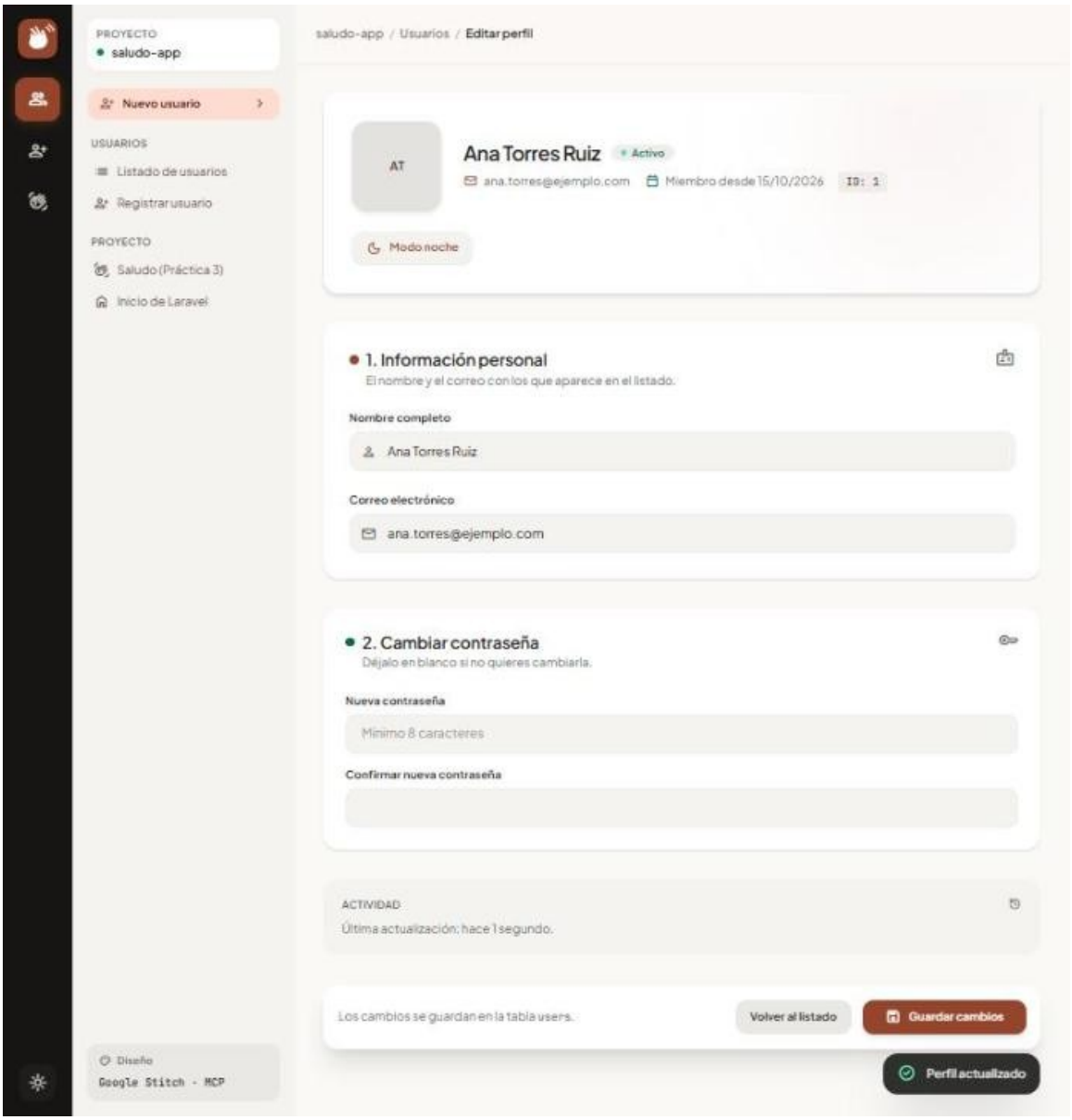
Captura 3. Validación del registro: nombre vacío, correo inválido y contraseña corta.



Captura 4. Listado después de registrar dos usuarios, con el aviso de confirmación.



Captura 5. Editar perfil de un usuario.



Captura 6. Perfil actualizado: el nombre cambió a "Ana Torres Ruiz".

Resultado

Lo que pide la práctica	Resultado
Agregar el MCP de Stitch en Visual Studio Code	Servidor `stitch` en `.vscode/mcp.json`, en estado Running con 15 herramientas; Copilot consultó el proyecto y sus pantallas por MCP (pasos 3 a 5)
Interfaz de **registro de usuario** en Laravel	`GET /usuarios/registro` → `usuarios/registro.blade.php`; guarda en la

Lo que pide la práctica	Resultado
	tabla `users` (capturas 2 y 3)
Interfaz de **listar usuarios** en Laravel	`GET /usuarios` → `usuarios/index.blade.php`, con búsqueda y paginación (capturas 1 y 4)
Interfaz de **editar perfil** en Laravel	`GET /usuarios/{id}/editar` → `usuarios/editar.blade.php`; cambia nombre, correo y contraseña (capturas 5 y 6)

La ruta `/saludo` de la Práctica 3 sigue funcionando igual.

Problemas encontrados y soluciones

Problema	Solución
No tenía proyectos en Stitch	Creé el proyecto "Panel de Usuarios Saludo-App" y generé ahí las tres pantallas
VS Code abrió la carpeta en Modo restringido y no mostraba Start	Marqué la carpeta como de confianza (Trust)
Error "Variable 'stitch-api-key' must be defined in an 'inputs' section"	Agregué el bloque `inputs` al `mcp.json`
Copilot intentaba abrir el HTML de cada pantalla como página web y salía en blanco	Le pedí el resumen solo con lo que devolvieron las herramientas MCP
El HTML de Stitch carga Tailwind desde un CDN y define los colores en un `<script>`	Los colores pasaron al `@theme` de `app.css` y se compilan con Vite
Las pantallas repetían el mismo menú lateral y barra superior	Un solo layout con componentes en `components/layout`
El proyecto está en inglés (`APP_LOCALE=en`) y los errores salían en inglés	Mensajes de validación en español y fechas con `locale('es')`
Sin `npm run dev` las páginas no cargaban los estilos	Estilos compilados en `public/build` e `iniciar.bat` para levantar el servidor

Conclusión

Conectar Stitch con VS Code por MCP permite que el agente consulte los diseños directamente, sin copiar HTML a mano: con el servidor en Running, Copilot listó el proyecto y sus pantallas en lenguaje natural. Aun así, el diseño no se usa tal cual: hay que adaptarlo a Laravel con layout, componentes, formularios, rutas y validación, y probar cada vista.

Lo más útil fue pasar primero el sistema de diseño al `@theme` y convertir una pantalla por petición, porque así las tres vistas quedaron consistentes. También aprendí a proteger la API key con `${input:...}` en lugar de escribirla en el archivo.